

On "Leave One Out" Shrinkage

Ian Atroshchenko

July 17 2026

In the current iteration of EMIT software, specifically, the "emit-ghg" publicly available repository and the ████████ code, we have reason to believe the Leave One Out (LOO) shrinkage (specifically the `fit_looshrinkage_alpha()` function) is improperly applied. The current implementation of `fit_looshrinkage_alpha()` is below:

```
1 def fit_looshrinkage_alpha(data, alphas, I_reg=[]):
2     # loocv shrinkage estimation via Theiler et al.
3     stability_scaling=100.0
4     nchan = data.shape[1]
5
6     nll = np.zeros(len(alphas))
7     n = data.shape[0]
8
9     X = data*stability_scaling
10    S = cov(X)
11    T = np.diag(np.diag(S)) if len(I_reg)==0 else cov(I_reg*
        stability_scaling)
12
13    nchanlog2pi = nchan*np.log(2.0*np.pi)
14    nll[:] = np.inf
15
16    # Closed form for leave one out cross validation error
17    for i,alpha in enumerate(alphas):
18        try:
19            # See Theiler, "The Incredible Shrinking Covariance
20                Estimator",
21            # Proc. SPIE, 2012. eqn. 29
22            beta = (1.0-alpha) / (n-1.0)
23            G_alpha = n * (beta*S) + (alpha*T)
24            G_det = scipy.linalg.det(G_alpha, check_finite=False)
25            if G_det==0:
26                continue
27            r_k = (X.dot(scipy.linalg.inv(G_alpha, check_finite=
28                False)) * X).sum(axis=1)
29            q = 1.0 - beta * r_k
30            nll[i] = 0.5*(nchanlog2pi+np.log(G_det))+1.0/(2.0*n) *
31                \
32                (np.log(q)+(r_k/q)).sum()
33        except np.linalg.LinAlgError:
34            logging.warning('looshrinkage encountered a LinAlgError
35                ')
```

```

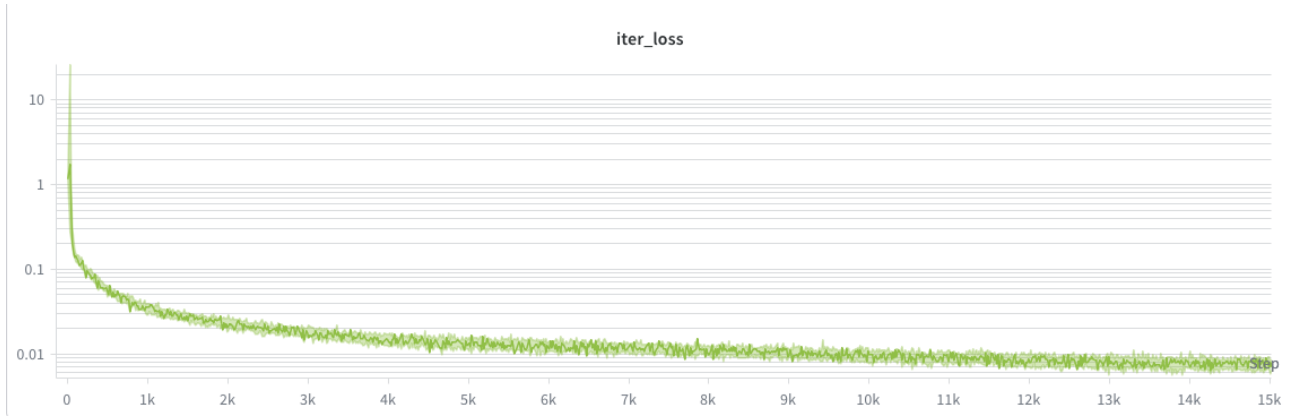
33     mindex = np.argmin(nll)
34     if nll[mindex] != np.inf:
35         alpha = alphas[mindex]
36     else:
37         mindex = -1
38         alpha = 0.0
39
40     return alpha

```

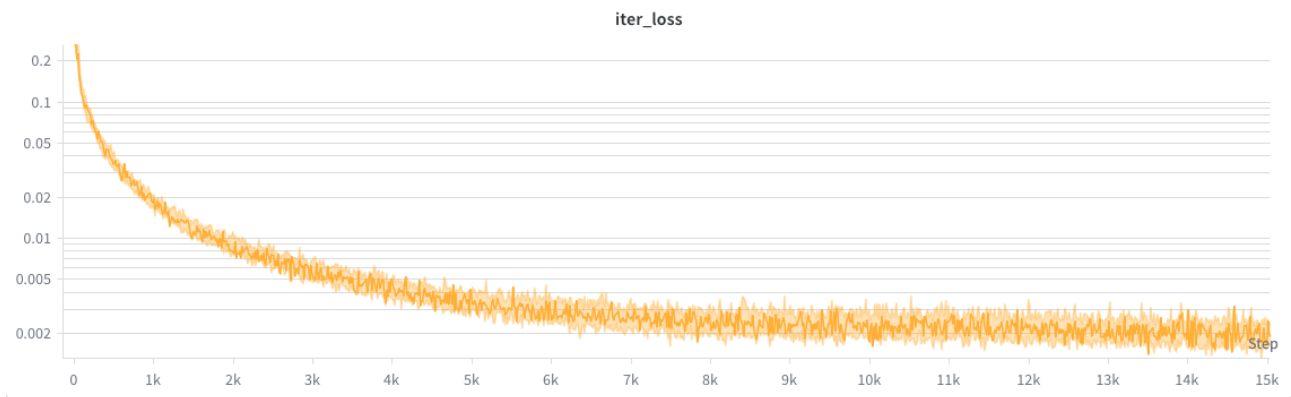
Recall the Theiler paper (2012), which is the basis for the current implementation of LOO shrinkage. Theiler 2012 explicitly states that LOO shrinkage relies on the input being assumed to be a "zero-mean Gaussian, i.e. that $\mu = 0$ ", and the code explicitly references Equation 29 (line 20) which falls under this assumption. However, nowhere in the software is the radiance input mean subtracted, e.g. in an expression such as $X = X - \text{np.mean}(X, \text{axis}=0)$, which results in a $\mu > 0$ that biases the output of `fit_looshrinkage_alpha()`. While the original covariance matrix S (line 10) is indeed mean centered due to the implementation of `np.cov()`, the r_k calculation on line 26 uses the non mean subtracted X directly, resulting in a Mahalanobis distance $r = x^T G^{-1} x$ that uses two different reference frames, a centered input G^{-1} derived from S and a non-centered input x^T and x . As a result, we propose a fix to mean subtract X before performing the alpha calculations, turning line 9 from `X = data * stability_scaling` to `X = (data - np.mean(data, axis=0))*stability_scaling`.

We have already implemented and used mean centered LOO shrinkage covariance matrices with success. In an implementation of a spectral transformer methane regression model that used whitened input directly, whitening with mean centered LOO shrinkage matrices achieved a test $r^2 = 0.95$. Training the exact same model without mean centering LOO shrinkage covariance matrices resulted in a test loss that was an order of magnitude greater than its mean centered counterpart and a slower learning process (Figure 1), along with a test $r^2 = 0.81$.

The exploration of covariance shrinkage for future EMIT mission products has the potential to yield better MF and DiffMF results. Covariance shrinkage is also important for the whitening operation through its usage of Cholesky factorization, which is currently used in ongoing projects that seek to improve the analysis of EMIT data. Because of this and our experimental results, we recommend the `fit_looshrinkage_alpha()` function to be modified as described.



(a) First 15k steps of a whitened regressor trained on non mean centered LOO shrinkage matrices. Note the end loss around 0.01.



(b) First 15k steps of a whitened regressor trained on mean centered LOO shrinkage matrices. Note the faster convergence and an end loss around 0.002, almost an order of magnitude lower than its counterpart.

Figure 1: Comparison of training processes between two identical spectral transformer regression models trained on different LOO shrinkage methodologies.